UNITY 6000.0.62F1 · COMPLETE OFFLINE DOCUMENTATION

Ludo Lane Developer Guide

Install. Edit. Reskin. Build your release.

szekipapa77@gmail.com



CHAPTER 01

Start here: Ludo Lane

Ludo Lane is a standalone editable Unity game template. Roll the die and select a token with a legal move to race your tokens home. Local computer guests compete on a Ludo board with party-specific player and token counts. Some parties add walls, special tiles or a head start. Use the displayed legal choices and check the current party rules; no online account or multiplayer server is supplied.

Includes 25 curated parties, source-backed generation, a game-specific visual editor and validation, 4 guided helpers, local saving and portrait/landscape scenes. Keep the required private game Kit and publisher-owned Szekipapa77Ads layer. Provider SDK binaries are excluded. There is no bundled HLK, purchases, multiplayer server or remote LiveOps backend. Import only one game Kit into a project because the Kit classes share names.

CHAPTER 02

Requirements, installation and first play

Use Unity **6000.0.62f1**, URP **17.0.4** and Unity UI **2.0.0**. Install matching Android SDK/NDK/OpenJDK and WebGL modules in Unity Hub. Retain the supplied manifest and ProjectSettings.

1. Extract the standalone project to a short writable local path, then open it in the matching editor. Wait for package import and compilation.
2. For native unitypackage import, create a matching URP project and use Assets > Import Package > Custom Package. Import every game, Kit, Szekipapa77Ads and WebGL-template entry.
3. Run Szekipapa77 > Ludo Lane > Setup. It configures rendering and player settings; review its changes when integrating with existing work.
4. Active Input Handling must support the classic Input Manager: choose Old or Both. New Input System only is insufficient.
5. Open `Assets/Games/LudoParty/Scenes/LudoParty_Mobile.unity` and press Play. Follow the level card and first-level instructions.
6. Stop Play mode before editing level assets. Keep all .meta GUIDs. The game uses classic Unity UI Text; do not assume an extra TextMesh Pro import is required.

CHAPTER 03

Project folders and exact source map

All game-relative paths below start at `Assets/Games/LudoParty`. Runtime UI is built in code; editing an unrelated static Canvas does not reskin it. Scripts contains game rules and views; Editor contains authoring tools; Kit supplies UI/audio/save/platform support; Resources/ludoparty contains level JSON, art, music and sound; Settings owns URP assets; Scenes contains both entry scenes.

- `Scripts/LpAI.cs`
- `Scripts/LpArt.cs`
- `Scripts/LpData.cs`
- `Scripts/LpGen.cs`
- `Scripts/LpLevels.cs`
- `Scripts/LpLogic.cs`
- `Scripts/LudoPartyGame.cs`
- `Scripts/LudoPartyGame.Debug.cs`
- `Scripts/LudoPartyGame.Helpers.cs`
- `Scripts/LudoPartyGame.Screens.cs`

Main flow: LudoPartyGame.cs. UI and popup consumers: LudoPartyGame.Screens.cs. Model/generator/loading: LpData.cs, LpGen.cs, LpLevels.cs, LpLogic.cs. Authoring: Editor/LudoPartyLevelEditor.cs. Shared ad settings live outside the game at Assets/Szekipapa77Ads/Resources/AdsSettings.asset.

CHAPTER 04

Gameplay, controls and progression

Roll the die and select a token with a legal move to race your tokens home. Local computer guests compete on a Ludo board with party-specific player and token counts. Some parties add walls, special tiles or a head start. Use the displayed legal choices and check the current party rules; no online account or multiplayer server is supplied.

The initial bank has 25 curated parties. Read the supplied loader/generator before changing continuation rules. Play the guided opening, ordinary progress, completion and retry paths; preserve saved indexes during a visual reskin. The Mobile scene is portrait and the WebGL-named scene is landscape, but the supplied Build menu selects portrait for both build targets. Select landscape explicitly in your Build Profile if desired. Test a near-startup scene replacement as well as a normal restart.

CHAPTER 05

Tutorials and every helper

Fresh local progress triggers the first-level guidance. Newly unlocked helpers have introduction and guided-use flows. Reset only a test save when reviewing tutorials. A free guided first use can differ from ordinary charge consumption. Exact names/unlocks/free charges: Scripts/LpData.cs; effect/help strings: Scripts/LpData.cs.

Helper	Unlock	Free charges	Action and effect
Reroll	3	3	Throw your dice again when the roll is no good.
Lucky Dice	6	3	Your next throw is a guaranteed 6.
Shield	10	3	Protect one token: it can't be captured for your next 3 turns.
Nudge	15	3	Add +1 to your roll before you move.

Activate the tool and perform its required target action: arming a tool alone does not demonstrate its effect. A successful use requires the stated preconditions from this game: show both activation and its actual effect. Exhausted charges may offer a rewarded ad; verify success/cancel/failure and grant a reward once only.

CHAPTER 06

Visual level editor: worked editing session

Open **Szekipapa77 > Ludo Lane > Level Editor**. Exact implementation: `Editor/LudoPartyLevelEditor.cs`.

Open the Ludo Lane editor and select an early party. Use Dup or Duplicate before editing. Change its visible name and one rule or board element with the exact controls below. Save explicitly and run the current entry through the editor validation and play controls. The per-game source-control table includes the current brush, geometry, target, rule and AI fields; do not treat hierarchy or resource IDs as display text.

1. Back up Resources/ludoparty/Levels/levels.json.
2. Duplicate a simple early level and edit one small element.
3. Use the game-specific validation controls and resolve every reported issue.
4. Save explicitly; stop and restart Play mode, then use Play this level or navigate to the edited index.
5. Check portrait and landscape and rebuild demo outputs after accepting data changes.

The exact current authoring controls, including tooltips and generator switches, follow. These are game-specific source references, not a generic editor feature list.

Source	Exact current code
`LudoPartyLevelEditor.cs:42`	`[MenuItem("Szekipapa77/Ludo Lane/Level Editor")]`
`LudoPartyLevelEditor.cs:45`	`[MenuItem("Szekipapa77/Ludo Lane/Validate Parties")]`
`LudoPartyLevelEditor.cs:113`	`if (GUILayout.Button(_dirty ? "Save *" : "Save", EditorStyles.toolbarButton, GUILayout.Width(70))) Save();`
`LudoPartyLevelEditor.cs:114`	`if (GUILayout.Button("Reload", EditorStyles.toolbarButton, GUILayout.Width(60)) && (!_dirty || EditorUtility.DisplayDialog("Reload", "Discard unsaved changes?", "Discard", "Cancel"))) Load();`
`LudoPartyLevelEditor.cs:119`	`if (GUILayout.Button("Validate party", EditorStyles.toolbarButton, GUILayout.Width(95))) ValidateOne(_data.selected);`
`LudoPartyLevelEditor.cs:120`	`if (GUILayout.Button("Validate all", EditorStyles.toolbarButton, GUILayout.Width(85))) ValidateAll();`

LudoPartyLevelEditor.cs:123	`if (GUILayout.Button("Play this party", EditorStyles.toolbarButton, GUILayout.Width(100))) PlayParty();`
LudoPartyLevelEditor.cs:152	`if (GUILayout.Button("Add")) { Rec("Add party"); int at = Mathf.Min(_data.selected + 1, File.matches.Count); File.matches.Insert(at, LpGenerator.Make(at + 1, 0.4f, Environment.TickCount & 0xffff)); _data.selected = at; _results.Clear(); }`
LudoPartyLevelEditor.cs:154	`if (GUILayout.Button("Dup")) { Rec("Duplicate party"); var c = Cur.Clone(); c.name += " copy"; File.matches.Insert(_data.selected + 1, c); _data.selected++; _results.Clear(); }`
LudoPartyLevelEditor.cs:155	`if (GUILayout.Button("Del") && EditorUtility.DisplayDialog("Delete party", $"Delete party {_data.selected + 1} ({Cur.name})?", "Delete", "Cancel")) { Rec("Delete party"); File.matches.RemoveAt(_data.selected); _data.selected = Mathf.Clamp(_data.selected, 0, File.matches.Count - 1); _results.Clear(); }`
LudoPartyLevelEditor.cs:156	`if (GUILayout.Button("Up") && _data.selected > 0) { Rec("Move party"); int s = _data.selected; (File.matches[s], File.matches[s - 1]) = (File.matches[s - 1], File.matches[s]); _data.selected--; _results.Clear(); }`
LudoPartyLevelEditor.cs:157	`if (GUILayout.Button("Down") && _data.selected < File.matches.Count - 1) { Rec("Move party"); int s = _data.selected; (File.matches[s], File.matches[s + 1]) = (File.matches[s + 1], File.matches[s]); _data.selected++; _results.Clear(); }`
LudoPartyLevelEditor.cs:161	`if (_results.TryGetValue(_data.selected, out var line)) EditorGUILayout.HelpBox(line, line.StartsWith("ERR", StringComparison.Ordinal) ? MessageType.Error : MessageType.Info);`
LudoPartyLevelEditor.cs:306	`if (GUILayout.Button("Clear tiles")) { Rec("Clear tiles"); Cur.tiles.Clear(); }`
LudoPartyLevelEditor.cs:307	`if (GUILayout.Button("Mirror to all 4 arms"))`
LudoPartyLevelEditor.cs:341	`string name = EditorGUILayout.TextField("Name", m.name);`

LudoPartyLevelEditor.cs:342	string venue = EditorGUILayout.TextField("Subtitle", m.venue);
LudoPartyLevelEditor.cs:343	int players = EditorGUILayout.IntSlider("Players (you + guests)", m.players, 2, 4);
LudoPartyLevelEditor.cs:344	int tokens = EditorGUILayout.IntSlider("Tokens each", m.tokens, 1, 4);
LudoPartyLevelEditor.cs:345	bool walls = EditorGUILayout.Toggle(new GUIContent("Walls", "Two tokens of one colour block passing"), m.rules.walls);
LudoPartyLevelEditor.cs:346	bool head = EditorGUILayout.Toggle(new GUIContent("Head start", "Everyone starts with one token out"), m.rules.headStart);
LudoPartyLevelEditor.cs:348	ui = EditorGUILayout.Popup("Unlocks helper", ui, UnlockNames);
LudoPartyLevelEditor.cs:349	Color tint = EditorGUILayout.ColorField("Board mat", Hex(m.tint));
LudoPartyLevelEditor.cs:350	string note = EditorGUILayout.TextField("Designer note", m.note);
LudoPartyLevelEditor.cs:362	EditorGUILayout.LabelField("Guest strength (avg)", m.Strength.ToString("0.00"));
LudoPartyLevelEditor.cs:380	string name = EditorGUILayout.TextField("Name", o.name);
LudoPartyLevelEditor.cs:381	float skill = EditorGUILayout.Slider(new GUIContent("Skill", "How often it plays its best scored move instead of a random legal one"), o.skill, 0, 1);
LudoPartyLevelEditor.cs:382	float aggr = EditorGUILayout.Slider(new GUIContent("Aggression", "Hunts captures"), o.aggression, 0, 1);
LudoPartyLevelEditor.cs:383	float caution = EditorGUILayout.Slider(new GUIContent("Caution", "Avoids cells rivals can reach, likes safe cells"), o.caution, 0, 1);
LudoPartyLevelEditor.cs:384	float block = EditorGUILayout.Slider(new GUIContent("Blocking", "Builds walls (Walls rule)"), o.blocking, 0, 1);

`LudoPartyLevelEditor.cs:385`	`float rush = EditorGUILayout.Slider(new GUIContent("Rush", "Races its leading token home"), o.rush, 0, 1);`
`LudoPartyLevelEditor.cs:386`	`float release = EditorGUILayout.Slider(new GUIContent("Release", "Brings tokens out of the yard"), o.release, 0, 1);`
`LudoPartyLevelEditor.cs:387`	`int avatar = EditorGUILayout.IntSlider("Avatar", o.avatar, 0, LpGenerator.AvatarCount - 1);`
`LudoPartyLevelEditor.cs:394`	`if (GUILayout.Button("Easy-going")) Preset(o, 0.05f);`
`LudoPartyLevelEditor.cs:395`	`if (GUILayout.Button("Friendly")) Preset(o, 0.35f);`
`LudoPartyLevelEditor.cs:396`	`if (GUILayout.Button("Sharp")) Preset(o, 0.7f);`
`LudoPartyLevelEditor.cs:397`	`if (GUILayout.Button("Champion")) Preset(o, 1f);`
`LudoPartyLevelEditor.cs:399`	`EditorGUILayout.LabelField("Strength", o.Strength.ToString("0.00"));`
`LudoPartyLevelEditor.cs:467`	`_genCount = EditorGUILayout.IntSlider("Count", _genCount, 1, 40);`
`LudoPartyLevelEditor.cs:468`	`_genReplace = EditorGUILayout.Toggle("Replace range", _genReplace);`
`LudoPartyLevelEditor.cs:469`	`_genStart = EditorGUILayout.IntField(_genReplace ? "Start party (1-based)" : "Ladder number of first", _genStart);`
`LudoPartyLevelEditor.cs:470`	`_genSeed = EditorGUILayout.IntField("Seed", _genSeed);`
`LudoPartyLevelEditor.cs:471`	`_dStart = EditorGUILayout.Slider("Start difficulty", _dStart, 0, 1);`
`LudoPartyLevelEditor.cs:472`	`_dEnd = EditorGUILayout.Slider("End difficulty", _dEnd, 0, 1);`
`LudoPartyLevelEditor.cs:473`	`_dSaw = EditorGUILayout.Slider("Sawtooth climb", _dSaw, 0, 0.2f);`

`LudoPartyLevelEditor.cs:474`	`_genBreather = EditorGUILayout.IntSlider("Breather every", _genBreather, 2, 10);`
`LudoPartyLevelEditor.cs:475`	`_dDrop = EditorGUILayout.Slider("Breather drop", _dDrop, 0, 0.5f);`
`LudoPartyLevelEditor.cs:476`	`_genValidate = EditorGUILayout.Toggle("Validate generated", _genValidate);`
`LudoPartyLevelEditor.cs:477`	`if (_genValidate) _genMatches = EditorGUILayout.IntSlider("Bot games per party", _genMatches, 4, 200);`
`LudoPartyLevelEditor.cs:486`	`if (GUILayout.Button(_genReplace ? $"Generate and replace parties {_genStart}-{_genStart + _genCount - 1}" : $"Generate and append {_genCount} parties")) Generate();`

CHAPTER 07

Generator, insertion and replacement

Use the actual generation panel and its fields listed below. Set a reproducible seed and a small count, then review the start/index and replace option before clicking Generate. The exact generation implementation is reproduced separately with line references so append, insert and replacement behavior can be distinguished. Save and reopen the data file to verify the intended entry count; runtime generated continuation is not a project-asset save.

Source	Exact current code
`LudoPartyLevelEditor.cs:31`	`bool _genReplace, _genValidate = true;`
`LudoPartyLevelEditor.cs:468`	`_genReplace = EditorGUILayout.Toggle("Replace range", _genReplace);`
`LudoPartyLevelEditor.cs:469`	`_genStart = EditorGUILayout.IntField(_genReplace ? "Start party (1-based)" : "Ladder number of first", _genStart);`
`LudoPartyLevelEditor.cs:486`	`if (GUILayout.Button(_genReplace ? $"Generate and replace parties {_genStart}-{_genStart + _genCount - 1}" : $"Generate and append {_genCount} parties")) Generate();`
`LudoPartyLevelEditor.cs:498`	`int index = (_genReplace ? _genStart : File.matches.Count + 1) + i;`
`LudoPartyLevelEditor.cs:512`	`if (_genReplace)`
`LudoPartyLevelEditor.cs:516`	`File.matches.RemoveRange(start, remove);`
`LudoPartyLevelEditor.cs:517`	`File.matches.InsertRange(start, made);`
`LudoPartyLevelEditor.cs:520`	`else { File.matches.AddRange(made); _data.selected = File.matches.Count - made.Count; }`

Worked extension: back up the bank, generate a small set at moderate difficulty using the exact per-game controls, validate them, then Save. Reopen the JSON and confirm the count changed. Do not assume every editor has the same insertion behavior: the actual implementation below distinguishes insert, append and replace. Runtime continuation does not automatically append new records to project JSON. Validation and bot scores help find problems but do not certify a pleasant human difficulty curve.

CHAPTER 08

Actual level JSON and identifiers

Canonical asset: `Assets/Games/LudoParty/Resources/ludoparty/Levels/levels.json`. Array order controls visible numbering. Preserve numeric/boolean types, root version fields where present and resource/save identifiers. The actual first record is shown below; use the editor for structured coordinates and block/word data.


```
{
 "name": "Backyard Picnic",
 "venue": "Your first party - one friendly guest",
 "tint": "#FFF5E6",
 "players": 2,
 "tokens": 2,
 "rules": {
   "walls": false,
   "headStart": false
 },
 "tiles": [],
 "unlock": "",
 "note": "curated d=0.00",
 "opponents": [
   {
     "name": "Pip",
     "avatar": 0,
     "aggression": 0.15,
     "caution": 0.1,
     "blocking": 0.17,
     "rush": 0.48,
     "release": 0.66,
     "skill": 0.05
   }
 ],
 "difficulty": 1.7,
 "winRate": 0.65
}
```


Root keys: version, matches. Inspect LpData.cs, LpGen.cs, LpLevels.cs, LpLogic.cs for field semantics and validation. Keep visible names/hints distinct from technical keys. The game-specific board, wave, card, opponent or question records are gameplay data; UI translation is separate.

CHAPTER 09

Reskin recipe: interface colours

Stop Play mode and open `Scripts/LpArt.cs`. Find `Ink`, currently `#2B1E4A`. Change that hex literal to `#42C9B0`, preserve its symbol/type, compile and reopen the scene. Review HUD, menus, helper cards, settings/results and both orientations. Art sprites and procedural fallback shapes can have additional colours; edit them deliberately.

Actual consumers of this symbol are listed first, then all current palette/hex assignments. Editor-only palettes do not control the runtime. Preserve state/goal contrast and rebuild after accepting a change.

Source	Exact current code
`LpArt.cs:11 / class/static declarations`	`Gold = Hex("#FFD166"), Ink = Hex("#2B1E4A"), Mint = Hex("#2EC4A0"), Coral = Hex("#FF6B6B"), Sky = Hex("#4C8DFF"), Sun = Hex("#FFC23C"),`

Source	Exact current code
`LpArt.cs:10 / class/static declarations`	`public static readonly Color NightTop = Hex("#4A2C6F"), NightBot = Hex("#1B1233"), Plum = Hex("#2B1E4A"), Cream = Hex("#FFF5E6"),`
`LpArt.cs:11 / class/static declarations`	`Gold = Hex("#FFD166"), Ink = Hex("#2B1E4A"), Mint = Hex("#2EC4A0"), Coral = Hex("#FF6B6B"), Sky = Hex("#4C8DFF"), Sun = Hex("#FFC23C"),`
`LpArt.cs:12 / class/static declarations`	`Lilac = Hex("#8E6CEF"), TextDark = Hex("#3A2A55"), TextSub = Hex("#7A6A93");`
`LpArt.cs:15 / class/static declarations`	`public static readonly Color[] SeatColor = { Hex("#FF6B6B"), Hex("#FFC23C"), Hex("#2EC4A0"), Hex("#4C8DFF") };`
`LpData.cs:58 / class/static declarations`	`public string tint = "#FFF5E6"; // board mat colour`
`LpGen.cs:15 / class/static declarations`	`public static readonly string[] Tints = { "#FFF5E6", "#FDEFF4", "#EEF6FF", "#EFFAF3", "#FFF8DC", "#F3EEFF", "#FFEFE6", "#EAF7F7" };`
`LudoPartyGame.cs:191 / void BuildChrome()`	`_fastBtn = HzIcons.IconButton(_hud, "\ue01f", Fast ? LpArt.Mint : HzUI.Hex("#5B4B8A"), () => ToggleFast(), 100); // fast_forward`
`LudoPartyGame.cs:210 / void ToggleFast()`	`_fastBtn.GetComponent<Image>().color = Fast ? LpArt.Mint : HzUI.Hex("#5B4B8A");`

`LudoPartyGame.cs:258 / void BuildBoard()`	`var tint = LpArt.Hex(_match != null ? _match.tint : "#FFF5E6");`
`LudoPartyGame.cs:261 / void BuildBoard()`	`var rim = HzUI.Panel("Rim", _boardStatic, LpArt.Hex("#6B4A9A"), 44); rim.raycastTarget = false;`
`LudoPartyGame.cs:307 / void BuildBoard()`	`if (tt == TileType.Star) cell.color = LpArt.Hex("#FFF1C2");`
`LudoPartyGame.cs:311 / void BuildBoard()`	`cell.color = tt == TileType.Popper ? LpArt.Hex("#FFE3D6") : tt == TileType.Slide ? LpArt.Hex("#E6DDFB") : LpArt.Hex("#DDF6EA");`
`LudoPartyGame.cs:570 / void ShowFace(int v)`	`var col = v == 6 ? LpArt.Hex("#E2463F") : LpArt.Plum;`
`LudoPartyGame.cs:592 / void SetDiceWaiting(bool on)`	`_diceGlow.color = _forced > 0 ? LpArt.Hex("#FFE27A") : LpArt.Gold;`
`LudoPartyGame.Helpers.cs:22 / class/static declarations`	`static readonly Color[] HelperColor = { HzUI.Hex("#FF8A5C"), HzUI.Hex("#FFC23C"), HzUI.Hex("#2EC4A0"), HzUI.Hex("#4C8DFF") };`
`LudoPartyGame.Helpers.cs:67 / void BuildHelperBar()`	`var bd = HzUI.Panel("Badge", h.btn.transform, HzUI.Hex("#FF4D6D"), 24);`
`LudoPartyGame.Helpers.cs:76 / void BuildHelperBar()`	`else { var li = HzIcons.Icon(lk.transform, HzIcons.Lock, (int)(sz * 0.4f), HzUI.Hex("#E6DDF5")); li.rectTransform.Place(new Vector2(0.5f, 0.5f), new Vector2(sz, sz * 0.5f), new Vector2(0, 12)); }`
`LudoPartyGame.Helpers.cs:129 / void RefreshHelpers()`	`h.badge.GetComponent<Image>().color = ch > 0 ? HzUI.Hex("#FF4D6D") : HzUI.Hex("#2EC4A0");`
`LudoPartyGame.Helpers.cs:172 / void UseHelper(string id)`	`if (_diceGlow) _diceGlow.color = LpArt.Hex("#FFE27A");`
`LudoPartyGame.Helpers.cs:266 / void OfferCharge(string id)`	`var m = Modal("Out of " + LpHelpers.Title(id), new Vector2(820, 660), HzUI.Hex("#4C8DFF"));`
`LudoPartyGame.Helpers.cs:320 / void ShowUnlock(string id, Action done)`	`var m = Modal("New helper!", new Vector2(860, 820), HzUI.Hex("#FF8A5C"));`
`LudoPartyGame.Helpers.cs:331 / void ShowUnlock(string id, Action done)`	`CardText(m, "+" + LpHelpers.FreeCharges + " free uses", 575, 600, 50, 34, HzUI.Hex("#D98A1E"), true);`

LudoPartyGame.Screens.cs:16 / class/static declarations	static readonly Color BtnGo = HzUI.Hex("#2EC4A0"), BtnAlt = HzUI.Hex("#6B4A9A"), BtnWarm = HzUI.Hex("#FF8A5C");
LudoPartyGame.Screens.cs:26 / class/static declarations	var card = HzUI.Panel("Card", dim.transform, HzUI.Hex("#FFF8F0"), 48);
LudoPartyGame.Screens.cs:100 / void Chips(ModalInfo m, List<string> labels, float y, float maxW)	var chip = HzUI.Panel("Chip", m.card, HzUI.Hex("#EDE3FA"), 22); chip.raycastTarget = false;
LudoPartyGame.Screens.cs:102 / void Chips(ModalInfo m, List<string> labels, float y, float maxW)	var t = HzUI.Label("T", chip.transform, l, 26, HzUI.Hex("#5B3F8C"), TextAnchor.MiddleCenter, false); t.fontStyle = FontStyle.Bold; t.rectTransform.Stretch();
LudoPartyGame.Screens.cs:151 / void ShowResult(bool won, int stars)	var st = HzIcons.Icon(m.card, HzIcons.Star, 110, i < stars ? HzUI.Hex("#FFC23C") : HzUI.Hex("#E2D8EA"));
LudoPartyGame.Screens.cs:181 / void ShowResult(bool won, int stars)	var bx = HzUI.Panel("Stat", m.card, HzUI.Hex("#F1E9FA"), 24); bx.raycastTarget = false;
LudoPartyGame.Screens.cs:192 / void ShowResult(bool won, int stars)	CardText(m, reward, y + 10, 800, 44, 30, HzUI.Hex("#D98A1E"));
LudoPartyGame.Screens.cs:214 / void ShowMenu()	sound = CardButton(m, SoundLabel(), HzUI.Hex("#4C8DFF"), 315, 262, 96, null, 36, -139);
LudoPartyGame.Screens.cs:216 / void ShowMenu()	music = CardButton(m, MusicLabel(), HzUI.Hex("#4C8DFF"), 315, 262, 96, null, 36, 139);
LudoPartyGame.Screens.cs:219 / void ShowMenu()	CardButton(m, "Restart party", HzUI.Hex("#F5A623"), 555, 540, 100, () => { CloseAllModals(); StopLoop(); HzSave.SetString(SaveKey, ""); HzPlatform.Interstitial(GameId, () => StartMatch(_matchNo, _testMatch ? _match : null)); }, 44);
LudoPartyGame.Screens.cs:220 / void ShowMenu()	CardButton(m, "Parties", HzUI.Hex("#8E6CEF"), 675, 540, 100, () => { CloseAllModals(); StopLoop(); ShowTitle(); ShowLadder(); }, 44);

`LudoPartyGame.Screens.cs:221 / void ShowMenu()`	`CardButton(m, "Main menu", HzUI.Hex("#8E6CEF"), 795, 540, 100, () => { CloseAllModals(); StopLoop(); ShowTitle(); }, 44);`
`LudoPartyGame.Screens.cs:296 / void ShowTitle()`	`title.AddShadow(new Color(0.1f, 0, 0.2f, 0.6f), 8); title.AddOutline(LpArt.Hex("#C2457A"), 4);`
`LudoPartyGame.Screens.cs:308 / void ShowTitle()`	`var tv = HzUI.Label("PVenue", tc.transform, match.venue, 28, HzUI.Hex("#D9C8F5"), TextAnchor.MiddleLeft, false);`
`LudoPartyGame.Screens.cs:337 / void ShowTitle()`	`var st = HzUI.Button("Stats", _title, "Stats", HzUI.Hex("#4C8DFF"), Color.white, ShowStats, 44);`
`LudoPartyGame.Screens.cs:343 / void ShowTitle()`	`var si = HzIcons.Icon(chip.transform, HzIcons.Star, 60, HzUI.Hex("#FFC23C")); si.rectTransform.Place(new Vector2(0, 0.5f), new Vector2(70, 70), new Vector2(50, 0));`
`LudoPartyGame.Screens.cs:379 / void ShowLadder()`	`var row = HzUI.Button("Row" + n, content, null, locked ? HzUI.Hex("#E6E0EC") : HzUI.Hex("#F6EEFB"), Color.white, () =>`
`LudoPartyGame.Screens.cs:386 / void ShowLadder()`	`var badge = HzUI.Panel("Num", row.transform, locked ? HzUI.Hex("#A79DB5") : LpArt.Hex(n % 2 == 0 ? "#8E6CEF" : "#FF6B6B"), 40); badge.raycastTarget = false;`
`LudoPartyGame.Screens.cs:393 / void ShowLadder()`	`var il = HzUI.Label("Info", row.transform, info, 24, HzUI.Hex("#7B3FA0"), TextAnchor.MiddleLeft, false);`
`LudoPartyGame.Screens.cs:399 / void ShowLadder()`	`var ul = HzUI.Label("Unlock", row.transform, "Unlocks " + LpHelpers.Title(t.unlock), 24, HzUI.Hex("#D98A1E"), TextAnchor.MiddleLeft, false);`
`LudoPartyGame.Screens.cs:418 / void ShowLadder()`	`var s = HzIcons.Icon(row.transform, HzIcons.Star, 44, i < stars ? HzUI.Hex("#FFC23C") : HzUI.Hex("#D8CDE3"));`
`LudoPartyGame.Screens.cs:440 / void ShowStats()`	`var m = Modal("Stats", new Vector2(880, 1160), HzUI.Hex("#4C8DFF"));`
`LudoPartyGame.Screens.cs:457 / void ShowStats()`	`var row = HzUI.Panel("Row", m.card, HzUI.Hex("#F3ECFA"), 22); row.raycastTarget = false;`

CHAPTER 10

Reskin recipe: visible menu and HUD text

Open `Scripts/LudoPartyGame.cs`, find the displayed literal `"Ludo Lane"`, and replace its displayed value with `"Ludo Lane Edition"`. Keep hierarchy names and technical IDs unchanged. In HzUI.Label/Button calls the first string may be an object name; change the displayed text argument. Read the method references below to distinguish construction from later refresh assignments.

Preserve interpolation braces, invariant number formatting, ON/OFF updates and line breaks. Update both initial and refresh paths, portrait and landscape. Exact names/unlocks/free charges: Scripts/LpData.cs; effect/help strings: Scripts/LpData.cs. Level names/hints remain separate JSON content. If a title appears in art, Title property or another screen, update those visible uses deliberately without renaming GameId, namespace, resources or save keys.

Hard-coded game strings do not automatically use shared localization. To translate the interface, route every display assignment through your translation table. Dictionaries, word banks and theme pools are independent gameplay assets and need separate validation. Test help, settings, results, tutorial and helper introductions after a fresh launch.

Source	Exact current code
`LpArt.cs:16 / class/static declarations`	`public static readonly string[] SeatName = { "Coral", "Sunny", "Mint", "Sky" };`
`LpArt.cs:39 / public static bool HasAvatar(int idx) => Load("Avatars/avatar_" + (((idx % LpGenerator.AvatarCount) + LpGenerator.AvatarCount) % LpGenerator.AvatarCount).ToString("00"));`	`public static Sprite HelperIcon(string id) => Load("Icons/icon_" + id);`
`LpData.cs:114 / public int TileCount(TileType t) { int n = 0; if (tiles != null) foreach (var x in tiles) if (x.type == (int)t) n++; return n; }`	`/// <summary>Helpers (boosters). Unlock parties come from the party data ("unlock" field). </summary>`
`LpGen.cs:215 / public static string Check(LpMatch m)`	`if (!string.IsNullOrEmpty(m.unlock) && Array.IndexOf(LpHelpers.Ids, m.unlock) < 0) return "unknown helper '" + m.unlock + "'";`
`LpLevels.cs:19 / class/static declarations`	`try { _file = ta ? JsonUtility.FromJson<LpFile> (ta.text) : null; } catch (Exception e) { Debug.LogWarning("[LudoParty] levels.json: " + e.Message); }`
`LudoPartyGame.cs:180 / void BuildChrome()`	`_hudTitle = HzUI.Label("Party", _hud, "", 46, Color.white, TextAnchor.MiddleLeft);`
`LudoPartyGame.cs:183 / void BuildChrome()`	`_hudSub = HzUI.Label("Sub", _hud, "", 26, LpArt.Gold, TextAnchor.MiddleLeft, false);`

`LudoPartyGame.cs:191 / void BuildChrome()`	`_fastBtn = HzIcons.IconButton(_hud, "\ue01f", Fast ? LpArt.Mint : HzUI.Hex("#5B4B8A"), () => ToggleFast(), 100); // fast_forward`
`LudoPartyGame.cs:198 / void BuildChrome()`	`_msgText = HzUI.Label("MsgText", _msgBg.transform, "", 34, Color.white, TextAnchor.MiddleCenter, false);`
`LudoPartyGame.cs:211 / void ToggleFast()`	`Toast(Fast ? "Fast guests: ON" : "Fast guests: OFF", LpArt.Gold, 1f, 0.75f);`
`LudoPartyGame.cs:219 / void RefreshHud()`	`_hudTitle.text = "Party " + _matchNo + ": " + _match.name;`
`LudoPartyGame.cs:220 / void RefreshHud()`	`_hudSub.text = string.Join(" · ", _match.TwistLabels());`
`LudoPartyGame.cs:487 / void BuildPlates()`	`pv.name = HzUI.Label("Name", bg.transform, NameOf(p), 34, Color.white, TextAnchor.MiddleLeft);`
`LudoPartyGame.cs:490 / void BuildPlates()`	`pv.sub = HzUI.Label("Sub", bg.transform, p == 0 ? LpArt.SeatName[seat] + " team" : SkillWord(Opp(p)), 22, LpArt.Lighter(SeatCol(seat), 0.35f), TextAnchor.MiddleLeft, false);`
`LudoPartyGame.cs:747 / IEnumerator TurnLoop()`	`bool helperChance = HelperUsable("reroll") || HelperUsable("nudge");`
`LudoPartyGame.Debug.cs:29 / public override string DebugCommand(string cmd)`	`case "charges": foreach (var id in LpHelpers.Ids) if (Prog.Has(id)) Prog.AddCharges(id, 9); SaveProg(); RefreshHelpers(); return "charges=" + string.Join(",", Prog.charges);`
`LudoPartyGame.Debug.cs:30 / public override string DebugCommand(string cmd)`	`case "helper": UseHelper(p.Length > 1 ? p[1] : "reroll"); return $"charges=[{string.Join(",", Prog.charges)}] forced={_forced} dice={_dice} shieldPick={_shieldPick}";`
`LudoPartyGame.Helpers.cs:22 / class/static declarations`	`static readonly Color[] HelperColor = { HzUI.Hex("#FF8A5C"), HzUI.Hex("#FFC23C"), HzUI.Hex("#2EC4A0"), HzUI.Hex("#4C8DFF") };`
`LudoPartyGame.Helpers.cs:47 / void BuildHelperBar()`	`h.btn = HzUI.Button("Helper_" + id, _hud, null, HelperColor[i], Color.white, () => UseHelper(id), 40, (int)(sz * 0.3f));`

`LudoPartyGame.Helpers.cs:48 / void BuildHelperBar()`	`h.btn.name = "Helper_" + id;`
`LudoPartyGame.Helpers.cs:63 / void BuildHelperBar()`	`h.label = HzUI.Label("Name", h.btn.transform, LpHelpers.Title(id), 24, Color.white, TextAnchor.MiddleCenter, false);`
`LudoPartyGame.Helpers.cs:70 / void BuildHelperBar()`	`h.badgeText = HzUI.Label("BadgeText", bd.transform, "", (int)(sz * 0.25f), Color.white); h.badgeText.rectTransform.Stretch();`
`LudoPartyGame.Helpers.cs:77 / void BuildHelperBar()`	`h.lockText = HzUI.Label("LockText", lk.transform, "", 22, LpArt.Gold, TextAnchor.MiddleCenter, false); h.lockText.fontStyle = FontStyle.Bold;`
`LudoPartyGame.Helpers.cs:121 / void RefreshHelpers()`	`h.lockText.text = at > 0 ? "Party " + at : "Soon";`
`LudoPartyGame.Helpers.cs:128 / void RefreshHelpers()`	`h.badgeText.text = ch > 0 ? ch.ToString() : "+";`
`LudoPartyGame.Helpers.cs:143 / void UseHelper(string id)`	`Toast(at > 0 ? LpHelpers.Title(id) + " unlocks at party " + at : "Coming soon", null, 1.4f, 0.75f);`
`LudoPartyGame.Helpers.cs:240 / void TryShield(int p, int t)`	`if (p != 0 || k < 0 || k > LpBoard.LastLoop || S.shield[t] > 0) { HzAudio.Play("wrong", 0.4f); Toast("Pick one of your tokens on the board", null, 1.2f, 0.75f); return; }`
`LudoPartyGame.Helpers.cs:257 / void FloatText(Vector2 at, string text, Color c)`	`var l = HzUI.Label("Float", Root, text, 64, c);`
`LudoPartyGame.Helpers.cs:266 / void OfferCharge(string id)`	`var m = Modal("Out of " + LpHelpers.Title(id), new Vector2(820, 660), HzUI.Hex("#4C8DFF"));`
`LudoPartyGame.Helpers.cs:270 / void OfferCharge(string id)`	`CardText(m, "Watch a short ad for one more " + LpHelpers.Title(id) + ".\nWinning a party also refills every helper by 1.", 380, 720, 120, 30, TextDark);`
`LudoPartyGame.Helpers.cs:271 / void OfferCharge(string id)`	`CardButton(m, "Watch ad +1", BtnGo, 490, 480, 104, () =>`
`LudoPartyGame.Helpers.cs:279 / void OfferCharge(string id)`	`Toast("+1 " + LpHelpers.Title(id), LpArt.Gold, 1.2f, 0.75f);`

LudoPartyGame.Helpers.cs:283 / void OfferCharge(string id)	CardButton(m, "No thanks", BtnAlt, 605, 360, 80, () => CloseModal(m), 36);
LudoPartyGame.Helpers.cs:331 / void ShowUnlock(string id, Action done)	CardText(m, "+" + LpHelpers.FreeCharges + " free uses", 575, 600, 50, 34, HzUI.Hex("#D98A1E"), true);
LudoPartyGame.Helpers.cs:332 / void ShowUnlock(string id, Action done)	CardButton(m, "Great!", BtnGo, 690, 420, 110, () => { CloseModal(m); done(); }, 52);
LudoPartyGame.Helpers.cs:346 / void ShowNewRule(string title, string text, Sprite art, Action done)	CardButton(m, "Got it", BtnGo, y + 290, 400, 110, () => { CloseModal(m); done(); }, 52);
LudoPartyGame.Helpers.cs:355 / void UpdateGuide()	foreach (var id in LpHelpers.Ids) if (Prog.Has(id) && !Seen("guide_" + id)) { _guide = id; break; }
LudoPartyGame.Screens.cs:32 / class/static declarations	var tl = HzUI.Label("Title", ribbon.transform, title, 60, Color.white); tl.rectTransform.Stretch(6);
LudoPartyGame.Screens.cs:76 / bool HasModal(string name) { foreach (var m in _modals) if (m.dim && m.dim.name == name) return true; return false; }	var t = HzUI.Label("Txt", m.card, text, size, color, align, display);
LudoPartyGame.Screens.cs:102 / void Chips(ModalInfo m, List<string> labels, float y, float maxW)	var t = HzUI.Label("T", chip.transform, l, 26, HzUI.Hex("#5B3F8C"), TextAnchor.MiddleCenter, false); t.fontStyle = FontStyle.Bold; t.rectTransform.Stretch();
LudoPartyGame.Screens.cs:132 / void ShowMatchIntro(Action done)	CardButton(m, "Let's party!", BtnGo, 770, 460, 104, () => { CloseModal(m); done(); }, 50);
LudoPartyGame.Screens.cs:139 / void ShowResult(bool won, int stars)	var winnerName = won ? "You" : NameOf(S.winner);
LudoPartyGame.Screens.cs:193 / void ShowResult(bool won, int stars)	CardButton(m, "Next party", BtnGo, y + 100, 520, 112, () => { CloseModal(m); HzPlatform.Interstitial(GameId, () => StartMatch(_testMatch ? Level : _matchNo + 1)); });
LudoPartyGame.Screens.cs:194 / void ShowResult(bool won, int stars)	CardButton(m, "Replay", BtnAlt, y + 225, 300, 90, () => { CloseModal(m); StartMatch(_matchNo, _testMatch ? _match : null); }, 40, -165);

`LudoPartyGame.Screens.cs:195 / void ShowResult(bool won, int stars)`	`CardButton(m, "Parties", BtnAlt, y + 225, 300, 90, () => { CloseModal(m); ShowTitle(); ShowLadder(); }, 40, 165);`
`LudoPartyGame.Screens.cs:200 / void ShowResult(bool won, int stars)`	`CardButton(m, "Try again", BtnGo, y + 100, 520, 112, () => { CloseModal(m); HzPlatform.Interstitial(GameId, () => StartMatch(_matchNo, _testMatch ? _match : null)); });`
`LudoPartyGame.Screens.cs:201 / void ShowResult(bool won, int stars)`	`CardButton(m, "Parties", BtnAlt, y + 220, 520, 96, () => { CloseModal(m); ShowTitle(); ShowLadder(); }, 44);`
`LudoPartyGame.Screens.cs:212 / void ShowMenu()`	`CardButton(m, "Resume", BtnGo, 190, 540, 110, () => CloseModal(m), 52);`
`LudoPartyGame.Screens.cs:214 / void ShowMenu()`	`sound = CardButton(m, SoundLabel(), HzUI.Hex("#4C8DFF"), 315, 262, 96, null, 36, -139);`
`LudoPartyGame.Screens.cs:215 / void ShowMenu()`	`sound.onClick.AddListener(() => { HzAudio.Muted = !HzAudio.Muted; sound.SetText(SoundLabel()); HzAudio.Play("toggle", 0.6f); });`
`LudoPartyGame.Screens.cs:216 / void ShowMenu()`	`music = CardButton(m, MusicLabel(), HzUI.Hex("#4C8DFF"), 315, 262, 96, null, 36, 139);`
`LudoPartyGame.Screens.cs:217 / void ShowMenu()`	`music.onClick.AddListener(() => { HzAudio.MusicMuted = !HzAudio.MusicMuted; music.SetText(MusicLabel()); HzAudio.Play("toggle", 0.6f); });`
`LudoPartyGame.Screens.cs:218 / void ShowMenu()`	`CardButton(m, "How to play", BtnWarm, 435, 540, 100, () => ShowRules(), 44);`
`LudoPartyGame.Screens.cs:219 / void ShowMenu()`	`CardButton(m, "Restart party", HzUI.Hex("#F5A623"), 555, 540, 100, () => { CloseAllModals(); StopLoop(); HzSave.SetString(SaveKey, ""); HzPlatform.Interstitial(GameId, () => StartMatch(_matchNo, _testMatch ? _match : null)); }, 44);`
`LudoPartyGame.Screens.cs:220 / void ShowMenu()`	`CardButton(m, "Parties", HzUI.Hex("#8E6CEF"), 675, 540, 100, () => { CloseAllModals(); StopLoop(); ShowTitle(); ShowLadder(); }, 44);`

`LudoPartyGame.Screens.cs:221 / void ShowMenu()`	`CardButton(m, "Main menu", HzUI.Hex("#8E6CEF"), 795, 540, 100, () => { CloseAllModals(); StopLoop(); ShowTitle(); }, 44);`
`LudoPartyGame.Screens.cs:224 / static string SoundLabel() => HzAudio.Muted ? "Sound: OFF" : "Sound: ON";`	`static string SoundLabel() => HzAudio.Muted ? "Sound: OFF" : "Sound: ON";`
`LudoPartyGame.Screens.cs:225 / static string MusicLabel() => HzAudio.MusicMuted ? "Music: OFF" : "Music: ON";`	`static string MusicLabel() => HzAudio.MusicMuted ? "Music: OFF" : "Music: ON";`
`LudoPartyGame.Screens.cs:249 / void ShowRules()`	`CardButton(m, "Got it", BtnGo, 1150, 400, 100, () => CloseModal(m), 46);`
`LudoPartyGame.Screens.cs:294 / void ShowTitle()`	`var title = HzUI.Label("Name", _title, "Ludo Lane", portrait ? 150 : 130, Color.white);`
`LudoPartyGame.Screens.cs:297 / void ShowTitle()`	`var tag = HzUI.Label("Tag", _title, "Roll, race and party with the guests", 34, LpArt.Gold, TextAnchor.MiddleCenter, false); tag.fontStyle = FontStyle.Bold;`
`LudoPartyGame.Screens.cs:305 / void ShowTitle()`	`var tl = HzUI.Label("PName", tc.transform, "Party " + cur + " " + match.name, 46, Color.white, TextAnchor.MiddleLeft);`
`LudoPartyGame.Screens.cs:308 / void ShowTitle()`	`var tv = HzUI.Label("PVenue", tc.transform, match.venue, 28, HzUI.Hex("#D9C8F5"), TextAnchor.MiddleLeft, false);`
`LudoPartyGame.Screens.cs:311 / void ShowTitle()`	`var tt = HzUI.Label("PTwist", tc.transform, string.Join(" · ", match.TwistLabels()), 26, LpArt.Gold, TextAnchor.MiddleLeft, false);`
`LudoPartyGame.Screens.cs:327 / void ShowTitle()`	`var play = HzUI.Button("Play", _title, cont ? "Continue" : "Play", LpArt.Coral, Color.white, () =>`
`LudoPartyGame.Screens.cs:335 / void ShowTitle()`	`var lad = HzUI.Button("Parties", _title, "Parties", BtnAlt, Color.white, ShowLadder, 44);`
`LudoPartyGame.Screens.cs:337 / void ShowTitle()`	`var st = HzUI.Button("Stats", _title, "Stats", HzUI.Hex("#4C8DFF"), Color.white, ShowStats, 44);`

`LudoPartyGame.Screens.cs:344 / void ShowTitle()`	`var sl = HzUI.Label("Stars", chip.transform, Prog.TotalStars + " / " + (Mathf.Max(1, Prog.unlocked - 1) * 3), 36, Color.white, TextAnchor.MiddleLeft); sl.rectTransform.Place(new Vector2(0, 0.5f), new Vector2(160, 60), new Vector2(92, 0), new Vector2(0, 0.5f));`
`LudoPartyGame.Screens.cs:349 / void ShowTitle()`	`help.name = "TitleHelp";`
`LudoPartyGame.Screens.cs:379 / void ShowLadder()`	`var row = HzUI.Button("Row" + n, content, null, locked ? HzUI.Hex("#E6E0EC") : HzUI.Hex("#F6EEFB"), Color.white, () =>`
`LudoPartyGame.Screens.cs:381 / void ShowLadder()`	`if (locked) { HzAudio.Play("wrong", 0.4f); Toast("Win party " + (tn - 1) + " to unlock"); return; }`
`LudoPartyGame.Screens.cs:388 / void ShowLadder()`	`HzUI.Label("N", badge.transform, n.ToString(), 46, Color.white).rectTransform.Stretch();`
`LudoPartyGame.Screens.cs:389 / void ShowLadder()`	`var nm = HzUI.Label("Name", row.transform, t.name, 40, locked ? TextSub : TextDark, TextAnchor.MiddleLeft);`
`LudoPartyGame.Screens.cs:393 / void ShowLadder()`	`var il = HzUI.Label("Info", row.transform, info, 24, HzUI.Hex("#7B3FA0"), TextAnchor.MiddleLeft, false);`
`LudoPartyGame.Screens.cs:399 / void ShowLadder()`	`var ul = HzUI.Label("Unlock", row.transform, "Unlocks " + LpHelpers.Title(t.unlock), 24, HzUI.Hex("#D98A1E"), TextAnchor.MiddleLeft, false);`
`LudoPartyGame.Screens.cs:425 / void ShowLadder()`	`HzUI.Label("T", nw.transform, "NEW", 26, Color.white).rectTransform.Stretch();`
`LudoPartyGame.Screens.cs:432 / void ShowLadder()`	`CardButton(m, "Close", BtnGo, h - 75, 400, 100, () => CloseModal(m), 46);`
`LudoPartyGame.Screens.cs:452 / void ShowStats()`	`("Helpers used", Stats.helpersUsed.ToString()),`
`LudoPartyGame.Screens.cs:463 / void ShowStats()`	`CardButton(m, "Close", BtnGo, y + 80, 400, 100, () => CloseModal(m), 46);`

CHAPTER 11

Artwork, fonts, audio and licences

Replace game images under Resources/ludoparty through Unity while retaining resource keys, .meta GUIDs, Sprite/UI import settings, alpha, pivots and slicing. Procedural graphics in the art/view sources may need separate colour changes. Replacing one PNG does not change every fallback. Keep third-party font and other licence notices with the corresponding shipped assets.

Effects are under Resources/ludoparty/Sfx. Preserve each actual load key and importer settings when replacing sounds. Supplied music: Resources/ludoparty/Music/party.ogg. Preserve that load key and verify loop/compression and the independent Music toggle. Kit UI uses supplied fonts; preserve actual licence notices and verify new glyphs.

CHAPTER 12

Local saves, reset and migration

HzSave wraps PlayerPrefs and flushes local state on focus loss/quit. WebGL persistence uses browser storage tied to the hosting origin. Clearing site data or changing origin starts a new local save; this is not cloud synchronization. Inspect HzSave/PlayerPrefs calls in the actual game source before altering progression, stars, helper charges or tutorial flags.

Verify fresh launch, a completed arena, helper use, sound options, focus loss and restart. Use a dedicated test profile for resets. Keep technical save keys stable during colour/text reskins. If changing schema or keys, implement and test a migration rather than silently discarding existing users' progress.

CHAPTER 13

Advertising integration and configuration

Asset: `Assets/Szekipapa77Ads/Resources/AdsSettings.asset`. Inspect backend, useGoogleTestIds and missingSdkFallback in the publisher-owned Ads configuration. Buyer projects include adapter/integration code and exclude external provider SDK binaries. WebGL and a missing-provider fallback simulate ads; these are not live commercial impressions.

For real Android ads install the supported provider SDK from its official source into your working project, enter your own app/placement IDs and use test ads first. Verify banner/interstitial/rewarded callbacks on supported hardware. For production use your own IDs and disable simulated missing-SDK fallback. Build tooling rejects unsafe production fallback settings. No IAP, remote backend or HLK is included in this publisher edition.

CHAPTER 14

Android build and release

Install Unity 6000.0.62f1 Android modules through Hub. Select Android, run Setup and review application ID `com.szekipapa77.ludoparty`, version, scene orientation and signing. The build command writes `Builds/Android/ludoparty.apk`. Confirm BuildResult.Succeeded and the actual output file; intermediate report totals are not the final APK size.

Use your own keystore/signing and distribution settings. A test-signed demo is not a signed store release. A provider-free build simulates ads; a provider test-demo does not prove production ad-account configuration. Test real hardware touch, safe areas, focus/audio, save/relaunch and reward callbacks. Keep passwords, keystores and private service credentials out of buyer packages.

CHAPTER 15

WebGL build and hosting

Install matching WebGL support. Run Setup and Build WebGL; output is `Builds/WebGL/ludoparty/`. Serve index.html with all loader/framework/data/WASM/StreamingAssets files over HTTP(S), not file://. Match compression, Content-Encoding and MIME types to the host. Outer page sizing and actual scene orientation are different settings.

The publisher's playable demo is hosted on HyperCodeStore. Upload the actual runtime files rather than merely changing a URL. Test a fresh anonymous load, real pointer controls, audio after interaction, saved progress/reload and phone/desktop sizing. Rebuild and rehost when code/assets change. A successful HTTP page alone is not gameplay verification.

CHAPTER 16

Troubleshooting

Compile errors: restore matching packages and the full private Kit/Ads layer; only one game Kit belongs in a project. **Pink/invisible rendering:** run Setup and inspect Graphics and every Quality URP assignment. **No input:** classic input handling must be enabled; check canvas focus and blocking modals. **Missing resources:** restore exact paths, .meta GUIDs and importer settings.

Edits do not appear: stop Play, Save in the level editor, reload the canonical bank and check current saved level. **Text reverts:** update construction and refresh assignments from the exact table. **No live ads:** provider binaries are deliberately excluded; inspect your installed provider and fallback mode. **Scene reload exception:** retain owner-lifetime and generation guards on delayed callbacks. **WebGL fails:** inspect browser Console, compression headers and missing payloads.

CHAPTER 17

Support and final release checklist

Publisher: Szekipapa77. Support: szekipapa77@gmail.com. Include game title, exact Unity version, platform, first Console error and reproduction steps. Never send passwords/tokens.

1. Clean-import the standalone project and native package.
2. Play both scenes and replace a scene near startup.
3. Review tutorial, normal completion/retry and every helper activation plus effect.
4. Validate editor changes and test reskin text/colours in both layouts.
5. Rebuild affected Android/WebGL outputs.
6. Verify complete public APK bytes, actual browser demo, documentation and video links.
7. Keep licences and exclude all internal prompts, planning/capture/build/QA tooling from buyers.

Ludo Lane · Szekipapa77 · Guide revision 2026-11-05